

Instituto Federal

- **Campus Goiânia**

Bacharelado em Sistemas de Informação

POO II

Prof. Dory Gonzaga Rodrigues

**Ementa****INTRODUÇÃO**

- Componentes
- Gerenciamento de Dependência
- JPA





Componentes

O que é um componente?

Componente é uma parte do código de um sistema (módulo) totalmente desacoplado e sem dependências, utilizados em linguagens orientadas a objetos na solução de um problema maior dividindo-os em diversas soluções menores que, em conjunto, resolvem o objetivo principal do sistema.

Objetivo do uso de componentes?

O objetivo principal do uso de componentes em um projeto JAVA é o reaproveitamento. Os componentes podem ser utilizados em vários projetos que sejam semelhantes, evitando duplicação de código e outros problemas.

No Java, o reuso de código (componentes) pode ser feito através da utilização de pacotes JAR. Pacote JAR são arquivos compactados que contêm classes já desenvolvidas e que podem ser adicionados a um novo projeto.



Gerenciamento de Dependência

Gerenciamento de Dependência

Nos projetos de grande porte é muito complicado e trabalhoso manter uma configuração manual desse **conjunto de componentes (biblioteca)**, especialmente do ponto de vista de manutenção da aplicação, o que pode gerar um oneroso trabalho na busca e inclusão de todos os arquivos necessários para o funcionamento do sistema.

Visando resolver esse problema, foram criadas algumas ferramentas chamadas de **Gerenciadores de dependências que têm como objetivo gerenciar esse conjunto de componentes (biblioteca) externos ao projeto.**

A ferramenta que iremos utilizar no Gerenciamento de dependências:

MAVEN



Gerenciamento de Dependência

Apache Maven

Apache Maven é uma ferramenta de gerenciamento e de informações de projetos de software.

O Maven pode gerenciar a construção, o relatório e a documentação de um projeto a partir de uma informação central, localizada em um arquivo XML denominado **pom.xml**, onde contém todas as configurações necessárias para rodar a aplicação.

POM significa "Modelo de projeto de objeto"

O POM contém **todas as informações necessárias sobre um projeto**, bem como configurações de plug-ins (bibliotecas) a serem usados durante o processo de construção. É a manifestação declarativa de "quem", "o quê" e "onde", enquanto o ciclo de vida da construção é o "quando" e "como".

<https://maven.apache.org/index.html>



Gerenciamento de Dependência

Apache Maven

Vantagem:

- separa o código fonte dos arquivos de configuração, documentação e dependências.
- Conta com vários plugins que facilitam a geração e gerenciamento dos projetos, criando uma visão do projeto como uma entidade única que gera um artefato apenas.

Maven é

- É uma estrutura de gerenciamento de projetos.
- É uma ferramenta de Build e um Framework baseado em script
- É um conjunto de padrões disponível em um repositório
- É um pedaço de software usado para gerenciar e descrever projetos.
- Ele define um ciclo de vida padrão para construção, teste e implantação de artefatos de projeto.



Gerenciamento de Dependência

Arquivo pom.xml

Maven

- Um projeto Maven sempre deve conter um arquivo pom.xml.
- POM contém todas as peças importantes de informações sobre um projeto como:
 - * descrição do projeto
 - * versão
 - * informações de distribuição
 - * dependências, etc.
- É essencialmente uma conta única para encontrar qualquer coisa relacionada a um projeto.
- POM é a unidade básica de trabalho no Maven:
 - cada tarefa a ser realizada será conduzida com base nas informações encontradas nele.

Gerenciamento de Dependência

Arquivo pom.xml

```
https://maven.apache.org/xsd/maven-4.0.0.xsd (xsi:schemaLocation)
1<project xmlns="http://maven.apache.org/POM/4.0.0"
2  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
3  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.org/xsd/maven-4.0.0.xsd">
4  <modelVersion>4.0.0</modelVersion>
5  <groupId>br.edu.ifgoias</groupId>
6  <artifactId>aulajpa</artifactId>
7  <version>0.0.1-SNAPSHOT</version>
8  <properties>
9    <maven.compiler.source>17</maven.compiler.source>
10    <maven.compiler.target>17</maven.compiler.target>
11  </properties>
12  <dependencies>
13    <!-- https://mvnrepository.com/artifact/org.hibernate.orm/hibernate-core -->
14    <dependency>
15      <groupId>org.hibernate.orm</groupId>
16      <artifactId>hibernate-core</artifactId>
17      <version>7.0.3.Final</version>
18    </dependency>
19    <!-- https://mvnrepository.com/artifact/org.postgresql/postgresql -->
20    <dependency>
21      <groupId>org.postgresql</groupId>
22      <artifactId>postgresql</artifactId>
23      <version>42.7.7</version>
24    </dependency>
25  </dependencies>
26</project>
```

Versão do Java que
deve ser utilizada

Hibernate

Driver do Postgresql



Programação OO x Bancos Relacionais

Na Programação Orientada a Objeto, quando falamos de persistência dos objetos, temos o desafio de mapear os objetos para o modelo entidade-relacionamento do banco de dados.

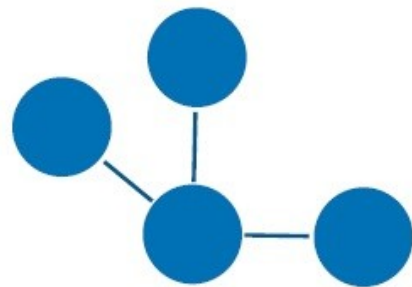
Este processo é extremamente complexo:

- Demanda um grande quantidade tempo dos programadores
- Dificuldade em mapear herança, agregações e composições de uma forma eficiente
- Diversos problemas relacionados ao desempenho
- Redução da produtividade e da flexibilidade pela utilização de SQL específica a um determinado banco de dados
- Dilema entre mapear as regras de negócio no banco de dados e na aplicação
- Embora existam banco de dados orientados à objetos, banco de dados relacionais são o padrão para o mercado



Programação OO x Bancos Relacionais

PROBLEMA

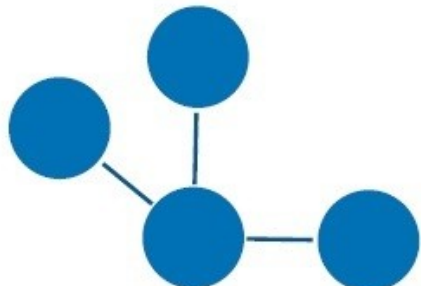


Programação Orientada
à Objetos

≠



SOLUÇÃO



Programação Orientada
à Objetos

Java Persistence API

JPA





JPA

Java Persistence API - JPA

Java Persistence API é uma API padrão da linguagem Java que descreve uma interface comum para frameworks de persistência de dados.

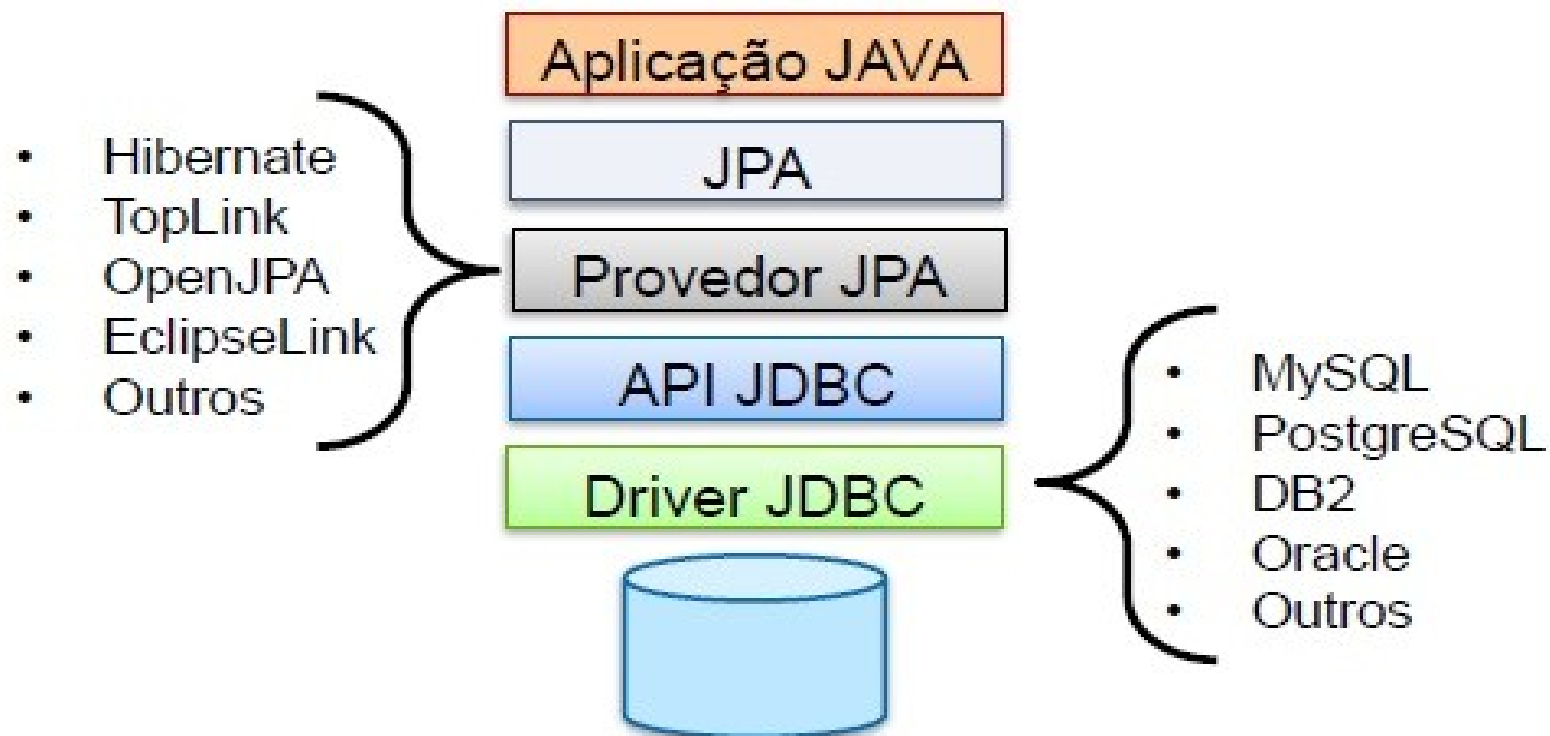
A JPA define um meio de mapeamento objeto-relacional para objetos Java simples e comuns, denominados beans de entidade ou modelo de dados.

- É **uma especificação** (abstração) em cima do JDBC que torna possível a persistência de dados sem utilizar um SQL específico do banco
- Utiliza **anotações** para o mapeamento dos elementos
- As classes e anotações da API JPA estão no **pacote jakarta.persistence**



JPA

Java Persistence API - JPA





JPA

Java Persistence API - JPA

Principais Componentes

- **Object-Relational Mapping** (ORM) ou Mapeamento Objeto Relacional (MOR)
Faz o mapeamento entre as classes (Objeto) e as tabelas do banco (Relacional)
- **EntityManager**
Uma API que permite desempenhar **operações CRUD** sobre um banco de dados
- **Java Persistence Query Language** (JPQL)
Uma linguagem de consulta Orientada à Objetos



JPA

Java Persistence API - JPA

Configurando o JPA com as propriedades do banco

- Arquivo: [persistence.xml](#)

Este arquivo conterá as configurações utilizadas pela JPA para a conexão do banco de dados, permitindo que a aplicação consiga realizar o acesso a dados.

- No Maven, crie o arquivo dentro da [pasta](#) de nome [META-INF](#). Esta pasta também deverá ser criada (caminho [“src/main/resources”](#))
- As classes [EntityManagerFactory](#) e [EntityManager](#) da JPA vão procurá-lo para fazer uso das configurações informadas ali.



JPA

Java Persistence API - JPA

Configurando o JPA com as propriedades do banco: [persistence.xml](#)

```
https://jakarta.ee/xml/ns/persistence/persistence_3_0.xsd (xsi:schemaLocation with catalog)
1 <?xml version="1.0" encoding="UTF-8"?>
2 <persistence xmlns="https://jakarta.ee/xml/ns/persistence"
3     xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
4     xsi:schemaLocation="https://jakarta.ee/xml/ns/persistence
5         https://jakarta.ee/xml/ns/persistence/persistence_3_0.xsd"
6         version="3.0">
7     <persistence-unit name="aulajpa" transaction-type="RESOURCE_LOCAL">
8         <class>aulajpa.Aluno</class>
9         <class>aulajpa.Curso</class>
10        <properties>
11            <property name="jakarta.persistence.jdbc.driver" value="org.postgresql.Driver" />
12            <property name="jakarta.persistence.jdbc.url" value="jdbc:postgresql://localhost/jpa" />
13            <property name="jakarta.persistence.jdbc.user" value="postgres" />
14            <property name="jakarta.persistence.jdbc.password" value="aluno" />
15            <property name="hibernate.dialect" value="org.hibernate.dialect.PostgreSQLDialect" />
16            <property name="hibernate.hbm2ddl.auto" value="validate" />
17            <property name="hibernate.show_sql" value="false" />
18            <property name="hibernate.format_sql" value="false" />
19        </properties>
20    </persistence-unit>
21 </persistence>
```



Mapeamento Objeto-Relacional (MOR)

A propriedade `hibernate.hbm2ddl.auto`

Serve para definir como o Hibernate vai lidar com o schema do banco de dados (ou seja, a estrutura, não os dados) no momento de inicialização da aplicação.

Valor	O que faz	Uso típico
none	Não faz nada	Produção
validate	Valida o schema, sem alterar	Produção segura
update	Atualiza schema sem apagar dados	Desenvolvimento leve
create	Apaga e cria esquema (ao iniciar)	Testes rápidos
create-drop	Apaga e cria esquema (início e fim)	Testes, desenvolvimento



Mapeamento Objeto-Relacional (MOR)

Entidade

Quando mapeamos **objetos de dados** para um BD relacional, para persisti-los ou consultá-los, dizemos que o objeto é uma **entidade**

- Objetos são apenas mantidos em memória e
- Entidades são objetos mantidos na memória durante algum intervalo de tempo e, então persistidos no banco de dados;
- Entidades, uma vez persistida no banco, podem ser consultadas através da JPQL

Atenção: Como ainda não estamos usando o Spring Boot, temos que registrar todas as entidades no `persistence.xml`

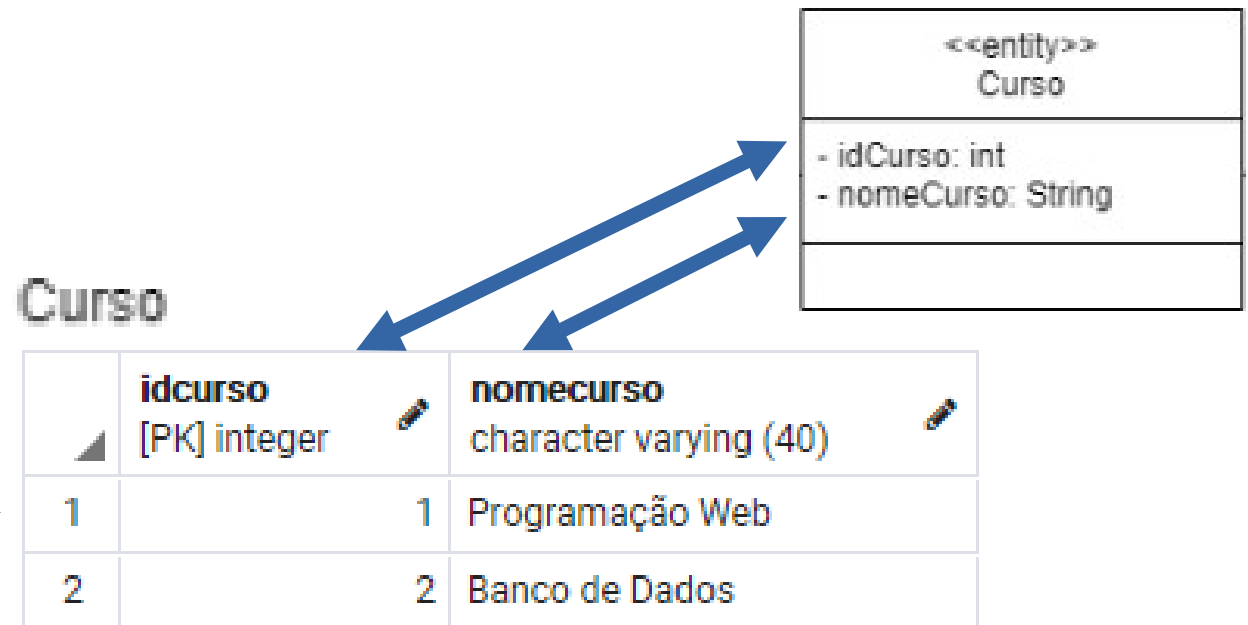
```
<persistence-unit name="aulajpa" transaction-type="RESOURCE_LOCAL">
    <class>aulajpa.Aluno</class>
    <class>aulajpa.Curso</class>
    <!-- Outras classes -->
    ...
</persistence-unit>
```



Mapeamento Objeto-Relacional (MOR)

Entidade

- A **classe** da entidade representa uma **tabela** do banco de dados



- Um **objeto** da entidade representa **uma linha na tabela**



JPA

Mapeamento Objeto-Relacional (MOR)

Anotação JPA para Entidade

Regras:

- A entidade deve ser anotada com `@javax.persistence.Entity`
- A anotação `@javax.persistence.Id` deve ser usada para criar uma chave primária simples
- A classe entidade deve possuir um construtor sem argumentos o qual deve ser `public` ou `protected`
- A classe entidade não pode ser um `enum` ou `interface`, ou seja, apenas `class` é permitido.
- A classe entidade não deve ser `final`.
- Nenhum dos métodos ou variáveis de instância pode ser `final`

JPA

Mapeamento Objeto-Relacional (MOR)

Anotação JPA para Entidade

Importando os pacotes necessários

Objetos da classe representam uma **linha** no banco.

@Id indica que a propriedade é a chave primária da tabela/entidade

@GeneratedValue indica que a propriedade será gerada pelo banco

Indica determinadas propriedades da coluna

```
1 package aulajpa;
2
3 import java.io.Serializable;
4 import java.util.ArrayList;
5 import java.util.List;
6
7 import jakarta.persistence.Column;
8 import jakarta.persistence.Entity;
9 import jakarta.persistence.GeneratedValue;
10 import jakarta.persistence.GenerationType;
11 import jakarta.persistence.Id;
12 import jakarta.persistence.JoinTable;
13 import jakarta.persistence.JoinColumn;
14 import jakarta.persistence.ManyToMany;
15
16 @Entity
17 public class Curso implements Serializable{
18     private static final long serialVersionUID = 1L;
19
20     @Id
21     @GeneratedValue (strategy = GenerationType.IDENTITY)
22     private Integer idcurso;
23
24     @Column (name = "nomecurso", nullable = false)
25     private String nomecurso;
```



JPA

Mapeamento Objeto-Relacional (MOR)

Anotação JPA para Entidade

Se as entidades respeitarem as **convenções**, o provedor de persistência pode fazer o **mapeamento automaticamente**, são elas:

- O nome da classe é mapeado como o nome da tabela. Caso não seja o mesmo nome, fazemos uso da anotação **@Table**

@Table (name= "Curso")

- Os nomes dos atributos são mapeados como nome das colunas. Caso não seja o mesmo nome, fazemos uso da anotação **@Column**

@Column (name= "idCurso")

- Para mapear os tipos de dados, as mesmas regras do JDBC são válidas, como por exemplo **String** mapeia para **VARCHAR** (padrão de 255 caracteres)



JPA

Mapeamento Objeto-Relacional (MOR)

Anotação JPA para Entidade

- A propriedade de uma classe (Entidade) que é a chave primária da tabela anotamos com: `@Id`
- Valor da propriedade atribuído automaticamente pelo banco de dados anotamos com: `@GeneratedValue` com o atributo `strategy`.

Onde `strategy` pode ser:

`SEQUENCE` ou `IDENTITY`: define uma coluna do tipo auto increment

`TABLE`: provedor de persistência cria uma tabela para oferecer o valor

`AUTO`: a escolha da estratégia para geração do chave é feita automaticamente

- Exemplo

```
@Id
@GeneratedValue(strategy = GenerationType.IDENTITY)
private Integer idcurso;
```